Rapport de Travaux Pratiques

Web Service PHP & Application Android

Technologies utilisées

MySQL **PHP 8** **Android** **Volley** **Gson**

Module	Développement Mobile & Web Services
Niveau	Licence / Master
Sujet	Gestion des étudiants via API REST

1er mars 2026

Table des matières

1 Introduction et Objectifs **3**
1.1 Architecture globale . . . 3

2 Partie 1 — Base de Données MySQL **4**
2.1 Prérequis et environnement . . . 4
2.2 Création de la base de données . . . 4
2.3 Création de la table Etudiant . . . 4
2.4 Insertion de données de test . . . 5

3 Partie 2 — Web Service PHP **6**
3.1 Structure du projet PHP . . . 6
3.2 Classe de connexion PDO . . . 6
3.3 Classe Entité Etudiant . . . 7
3.4 Interface IDao . . . 7
3.5 Service EtudiantService . . . 8
3.6 Web Services PHP . . . 9
3.6.1 Service d'ajout — createEtudiant.php . . . 9
3.6.2 Service de lecture — loadEtudiant.php . . . 9
3.7 Test avec Advanced REST Client . . . 9
3.7.1 Test du service d'ajout (POST) . . . 9
3.7.2 Test du service de lecture (GET) . . . 10

4 Partie 3 — Application Android **11**
4.1 Création du projet . . . 11
4.2 Permission Internet . . . 11
4.3 Dépendances Gradle . . . 11
4.4 Styles et ressources . . . 12
4.4.1 Définition des styles . . . 12
4.4.2 Ressource tableau de villes . . . 12
4.5 Layout de l'activité AddEtudiant . . . 12
4.6 Classe Etudiant (bean Android) . . . 13
4.7 Activité AddEtudiant . . . 14
4.8 Configuration de la sécurité réseau (Android 9+) . . . 16
4.8.1 Fichier network_security_config.xml . . . 16
4.8.2 Déclaration dans AndroidManifest.xml . . . 16
4.9 Test et résultats . . . 16
4.9.1 Procédure de test . . . 16
4.9.2 Sortie Logcat attendue . . . 17

5 Challenge — Extension du Projet **18**
5.1 Activité de liste des étudiants 18
5.1.1 Web Service PHP — deleteEtudiant.php 18
5.1.2 Méthode deleteById dans EtudiantService 18
5.1.3 Adapter RecyclerView Android 18

6 Conclusion **21**

Chapitre 1

Introduction et Objectifs

Ce travail pratique a pour but de mettre en œuvre une architecture client-serveur complète autour d'une base de données de gestion des étudiants. Il s'articule en trois grandes parties complémentaires.

Objectifs pédagogiques

1. Créer et alimenter une base de données **MySQL** pour stocker les informations des étudiants.
2. Développer un **Web Service PHP** capable d'ajouter des enregistrements et de retourner les données au format **JSON**.
3. Construire une **application Android** qui consomme ce service via la bibliothèque **Volley**.
4. Utiliser **Gson** pour parser la réponse JSON et afficher les résultats dans les logs (Logcat).

1.1 Architecture globale

L'architecture mise en place suit le modèle **3-tiers** classique :

Couche		Technologie	Rôle
Présentation	→	Android (Java)	Interface utilisateur
Logique métier	→	PHP 8	Web Services REST
Données	→	MySQL / XAMPP	Persistance

La communication entre la couche Android et la couche PHP se fait via des requêtes **HTTP POST/GET**, les données étant échangées au format **JSON**.

Chapitre 2

Partie 1 — Base de Données MySQL

2.1 Prérequis et environnement

Avant de commencer, il est nécessaire de disposer de **XAMPP** installé sur la machine de développement. XAMPP fournit un environnement intégré comprenant Apache, MySQL et PHP. Les étapes préliminaires sont les suivantes :

1. Démarrer XAMPP et activer les modules **Apache** et **MySQL**.
2. Ouvrir un navigateur et accéder à `http://localhost/phpmyadmin`.

2.2 Création de la base de données

La base de données `school1` est créée avec la commande SQL suivante :

```sql
CREATE DATABASE school1;
```

Listing 2.1 – Création de la base de données

2.3 Création de la table Etudiant

La table `Etudiant` est conçue pour stocker les informations de base de chaque étudiant. La colonne `id` est définie comme clé primaire avec auto-incrémentation.

```sql
CREATE TABLE Etudiant (
  id     INT          AUTO_INCREMENT PRIMARY KEY,
  nom    VARCHAR(50),
  prenom VARCHAR(50),
  ville  VARCHAR(50),
  sexe   VARCHAR(10)
);
```

Listing 2.2 – Création de la table Etudiant

2.4 Insertion de données de test

Des enregistrements initiaux sont insérés afin de pouvoir tester les Web Services ultérieurement :

```sql
INSERT INTO Etudiant (nom, prenom, ville, sexe)
VALUES ('Lachgar', 'Mohamed', 'Rabat',      'homme'),
       ('Safi',    'Amine',   'Marrakech', 'homme');
```

Listing 2.3 – Insertion d'enregistrements tests

Vérification

Après l'exécution de ces requêtes, la table doit contenir 2 enregistrements. Il est possible de le vérifier avec `SELECT * FROM Etudiant;` dans phpMyAdmin.

Chapitre 3

Partie 2 — Web Service PHP

3.1 Structure du projet PHP

Le projet PHP est organisé selon une architecture en couches, placé dans le répertoire `C:\xampp\htdocs\projet`. Les dossiers suivants sont créés :

Dossier	Rôle
`classes/`	Entités métier (POPOs)
`connexion/`	Classe de connexion PDO
`dao/`	Interface DAO générique
`service/`	Logique métier et requêtes SQL
`controller/`	Contrôleurs (optionnel)
`ws/`	Points d'entrée des Web Services

3.2 Classe de connexion PDO

Le fichier `connexion/Connexion.php` gère la connexion à la base de données via **PDO** (PHP Data Objects), ce qui garantit une abstraction de la base et une meilleure sécurité grâce aux requêtes préparées.

```php
<?php
class Connexion {
    private $connexion;

    public function __construct() {
        try {
            $this->connexion = new PDO(
                "mysql:host=localhost;dbname=school1;charset=utf8",
                "root",
                ""
            );
            $this->connexion->setAttribute(
                PDO::ATTR_ERRMODE,
                PDO::ERRMODE_EXCEPTION
            );
```

```
        } catch (PDOException $e) {
            die('Erreur : ' . $e->getMessage());
        }
    }

    public function getConnexion() {
        return $this->connexion;
    }
}
?>
```

Listing 3.1 – connexion/Connexion.php

3.3 Classe Entité Etudiant

Le fichier `classes/Etudiant.php` représente l'entité métier. Il définit les propriétés et les accesseurs (getters) nécessaires au service.

```
<?php
class Etudiant {
    private $id, $nom, $prenom, $ville, $sexe;

    function __construct($id, $nom, $prenom, $ville, $sexe) {
        $this->id     = $id;
        $this->nom    = $nom;
        $this->prenom = $prenom;
        $this->ville  = $ville;
        $this->sexe   = $sexe;
    }

    function getNom()    { return $this->nom;    }
    function getPrenom() { return $this->prenom; }
    function getVille()  { return $this->ville;  }
    function getSexe()   { return $this->sexe;   }
}
?>
```

Listing 3.2 – classes/Etudiant.php

3.4 Interface IDao

L'interface `dao/IDao.php` définit le contrat que doit respecter tout service d'accès aux données. Elle impose les opérations CRUD standard.

```
<?php
interface IDao {
    function create($o);
    function delete($o);
    function update($o);
    function findAll();
```

```
    function findById($id);
}
?>
```

Listing 3.3 – dao/IDao.php

3.5 Service EtudiantService

Le fichier `service/EtudiantService.php` implémente l'interface `IDao`. Il contient toute la logique d'accès aux données : insertion et lecture depuis la base MySQL.

```
<?php
include_once '../classes/Etudiant.php';
include_once '../connexion/Connexion.php';
include_once '../dao/IDao.php';

class EtudiantService implements IDao {
    private $connexion;

    function __construct() {
        $this->connexion = new Connexion();
    }

    public function create($o) {
        $query = "INSERT INTO Etudiant (nom, prenom, ville, sexe)
                  VALUES (:nom, :prenom, :ville, :sexe)";
        $stmt = $this->connexion->getConnexion()->prepare($query);
        $stmt->execute([
            ':nom'    => $o->getNom(),
            ':prenom' => $o->getPrenom(),
            ':ville'  => $o->getVille(),
            ':sexe'   => $o->getSexe()
        ]);
    }

    public function findAllApi() {
        $req = $this->connexion->getConnexion()
                               ->query("SELECT * FROM Etudiant");
        return $req->fetchAll(PDO::FETCH_ASSOC);
    }

    // M thodes IDao non impl ment es dans ce TP
    public function delete($o)      {}
    public function update($o)      {}
    public function findAll()       {}
    public function findById($id)   {}
}
?>
```

Listing 3.4 – service/EtudiantService.php

3.6 Web Services PHP

3.6.1 Service d'ajout — createEtudiant.php

Ce service accepte une requête **POST**, crée un objet `Etudiant`, l'insère en base, puis retourne la liste complète au format JSON.

```php
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    include_once '../service/EtudiantService.php';
    extract($_POST);

    $es = new EtudiantService();
    $es->create(new Etudiant(1, $nom, $prenom, $ville, $sexe));

    header('Content-Type: application/json');
    echo json_encode($es->findAllApi());
}
?>
```

Listing 3.5 – ws/createEtudiant.php

3.6.2 Service de lecture — loadEtudiant.php

Ce service accepte une requête **GET** et retourne la liste complète des étudiants au format JSON, sans aucune modification en base.

```php
<?php
include_once '../service/EtudiantService.php';

$es = new EtudiantService();
header('Content-Type: application/json');
echo json_encode($es->findAllApi());
?>
```

Listing 3.6 – ws/loadEtudiant.php

3.7 Test avec Advanced REST Client

Avant d'intégrer les services dans l'application Android, ils peuvent être testés avec l'extension **Advanced REST Client** (ARC) ou **Postman**.

3.7.1 Test du service d'ajout (POST)

1. URL : `http://localhost/projet/ws/createEtudiant.php`

2. Méthode : **POST**
3. Corps (`x-www-form-urlencoded`) :

Clé	Valeur
`nom`	Dupont
`prenom`	Sara
`ville`	Casablanca
`sexe`	femme

Réponse JSON attendue :

```
[
  {"id":"1","nom":"Lachgar","prenom":"Mohamed","ville":"Rabat
     ","sexe":"homme"},
  {"id":"2","nom":"Safi","prenom":"Amine","ville":"Marrakech","
     sexe":"homme"},
  {"id":"3","nom":"Dupont","prenom":"Sara","ville":"Casablanca
     ","sexe":"femme"}
]
```

Listing 3.7 – Réponse du service createEtudiant

3.7.2 Test du service de lecture (GET)

1. URL : `http://localhost/projet/ws/loadEtudiant.php`
2. Méthode : **GET**
3. Cliquer sur *Send*

La réponse doit afficher l'ensemble des étudiants de la base sous forme de tableau JSON.

Chapitre 4

Partie 3 — Application Android

4.1 Création du projet

Un nouveau projet Android est créé dans Android Studio avec les paramètres suivants :

Paramètre	Valeur
Nom du projet	`projetws`
Type d'activité	Empty Activity
Langage	Java
API minimum	26 (Android 8.0)

4.2 Permission Internet

La permission Internet est déclarée dans `AndroidManifest.xml` avant la balise `<application>` :

```
<uses-permission android:name="android.permission.INTERNET" />
```

Listing 4.1 – Permission Internet dans AndroidManifest.xml

4.3 Dépendances Gradle

Les bibliothèques **Volley** et **Gson** sont ajoutées dans `build.gradle (Module: app)` :

```
dependencies {
    implementation 'androidx.appcompat:appcompat:1.6.1'
    implementation 'androidx.constraintlayout:constraintlayout
        :2.1.4'
    implementation 'com.android.volley:volley:1.2.1'
    implementation 'com.google.code.gson:gson:2.10.1'
    testImplementation 'junit:junit:4.13.2'
}
```

Listing 4.2 – build.gradle — section dependencies

Après modification, cliquer sur **Sync Now** pour télécharger les dépendances.

4.4 Styles et ressources

4.4.1 Définition des styles

Dans `res/values/styles.xml`, deux styles sont ajoutés pour harmoniser l'interface :

```
<style name="text">
    <item name="android:textSize">17sp</item>
    <item name="android:paddingBottom">10dp</item>
    <item name="android:textColor">@android:color/
       holo_green_dark</item>
</style>

<style name="input">
    <item name="android:paddingBottom">20dp</item>
</style>
```

Listing 4.3 – res/values/styles.xml

4.4.2 Ressource tableau de villes

Dans `res/values/arrays.xml`, le tableau de villes alimentant le `Spinner` est défini :

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="villes">
        <item>Rabat</item>
        <item>Casablanca</item>
        <item>Marrakech</item>
        <item>Fes</item>
        <item>Tanger</item>
        <item>Agadir</item>
    </string-array>
</resources>
```

Listing 4.4 – res/values/arrays.xml

4.5 Layout de l'activité AddEtudiant

Le fichier `res/layout/activity_add_etudiant.xml` définit l'interface graphique du formulaire d'ajout :

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".AddEtudiant">

```

```
    <TextView android:text="Nom :"     style="@style/text" />
    <EditText android:id="@+id/nom"    style="@style/input"
        android:inputType="textPersonName" />

    <TextView android:text="Prenom :" style="@style/text" />
    <EditText android:id="@+id/prenom" style="@style/input"
        android:inputType="textPersonName" />

    <TextView android:text="Ville :"  style="@style/text" />
    <Spinner android:id="@+id/ville"
        android:entries="@array/villes"
        style="@style/input" />

    <TextView android:text="Sexe :"   style="@style/text" />
    <RadioGroup android:orientation="horizontal">
        <RadioButton android:id="@+id/m" android:text="Homme"
            />
        <RadioButton android:id="@+id/f" android:text="Femme"
            />
    </RadioGroup>

    <Button
        android:id="@+id/add"
        android:text="Ajouter"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

</LinearLayout>
```

Listing 4.5 – res/layout/activity_add_etudiant.xml

4.6 Classe Etudiant (bean Android)

La classe `Etudiant.java` est placée dans un sous-package `beans`. Ses attributs correspondent exactement aux champs JSON retournés par le Web Service, ce qui permet à **Gson** de désérialiser automatiquement les objets.

```
package com.example.projetws.beans;

public class Etudiant {
    private int    id;
    private String nom, prenom, ville, sexe;

    public Etudiant() {}

    public Etudiant(int id, String nom, String prenom,
                    String ville, String sexe) {
        this.id     = id;
        this.nom    = nom;
        this.prenom = prenom;
        this.ville  = ville;
```

```java
        this.sexe   = sexe;
    }

    @Override
    public String toString() {
        return "Etudiant{"
            + "id="        + id
            + ", nom='"    + nom     + '\''
            + ", prenom='" + prenom  + '\''
            + ", ville='"  + ville   + '\''
            + ", sexe='"   + sexe    + '\''
            + '}';
    }
}
```

Listing 4.6 – beans/Etudiant.java

4.7 Activité AddEtudiant

La classe `AddEtudiant.java` orchestre la saisie du formulaire, l'envoi de la requête POST via **Volley** et le parsing de la réponse via **Gson**.

Adresse IP de l'émulateur

Dans l'émulateur Android, `10.0.2.2` est l'alias de `localhost` de la machine hôte. Il ne faut **jamais** utiliser `127.0.0.1` depuis l'émulateur.

```java
package com.example.projetws;

import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;
import com.android.volley.*;
import com.android.volley.toolbox.StringRequest;
import com.android.volley.toolbox.Volley;
import com.example.projetws.beans.Etudiant;
import com.google.gson.Gson;
import com.google.gson.reflect.TypeToken;
import java.lang.reflect.Type;
import java.util.*;

public class AddEtudiant extends AppCompatActivity
        implements View.OnClickListener {

    private EditText    nom, prenom;
    private Spinner     ville;
    private RadioButton m, f;
    private Button      add;
```

```
    private RequestQueue requestQueue;

    private static final String insertUrl =
        "http://10.0.2.2/projet/ws/createEtudiant.php";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_add_etudiant);

        nom    = findViewById(R.id.nom);
        prenom = findViewById(R.id.prenom);
        ville  = findViewById(R.id.ville);
        m      = findViewById(R.id.m);
        f      = findViewById(R.id.f);
        add    = findViewById(R.id.add);

        requestQueue = Volley.newRequestQueue(this);
        add.setOnClickListener(this);
    }

    @Override
    public void onClick(View v) {
        if (v == add) envoyerEtudiant();
    }

    private void envoyerEtudiant() {
        StringRequest request = new StringRequest(
                Request.Method.POST, insertUrl,
                response -> {
                    Log.d("RESPONSE", response);
                    Type type = new TypeToken<
                        Collection<Etudiant>>(){}.getType();
                    Collection<Etudiant> etudiants =
                        new Gson().fromJson(response, type);
                    for (Etudiant e : etudiants) {
                        Log.d("ETUDIANT", e.toString());
                    }
                },
                error -> Log.e("VOLLEY",
                    "Erreur : " + error.getMessage())
        ) {
            @Override
            protected Map<String, String> getParams() {
                String sexe = m.isChecked() ? "homme" : "femme"
                    ;
                Map<String, String> params = new HashMap<>();
                params.put("nom",    nom.getText().toString());
                params.put("prenom", prenom.getText().toString
                    ());
                params.put("ville",  ville.getSelectedItem().
```

```
                    toString());
                params.put("sexe",   sexe);
                return params;
            }
        };
        requestQueue.add(request);
    }
}
```

Listing 4.7 – AddEtudiant.java

4.8 Configuration de la sécurité réseau (Android 9+)

Depuis Android 9 (API 28), le trafic HTTP non chiffré est bloqué par défaut. Il est nécessaire de créer un fichier de configuration de sécurité pour autoriser la connexion locale.

4.8.1 Fichier network_security_config.xml

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
    <domain-config cleartextTrafficPermitted="true">
        <domain includeSubdomains="true">10.0.2.2</domain>
    </domain-config>
</network-security-config>
```

Listing 4.8 – res/xml/network_security_config.xml

4.8.2 Déclaration dans AndroidManifest.xml

```
<application
    android:networkSecurityConfig="@xml/network_security_config
       "
    android:usesCleartextTraffic="true"
    android:label="@string/app_name"
    ... >
    ...
</application>
```

Listing 4.9 – AndroidManifest.xml — balise application

4.9 Test et résultats

4.9.1 Procédure de test

1. S'assurer que **Apache** et **MySQL** sont démarrés dans XAMPP.
2. Lancer l'application sur l'émulateur Android.

3. Remplir le formulaire et cliquer sur **Ajouter**.
4. Ouvrir **Logcat** dans Android Studio (filtre : `ETUDIANT` ou `RESPONSE`).

4.9.2 Sortie Logcat attendue

```
D/RESPONSE: [
  {"id":"1","nom":"Lachgar","prenom":"Mohamed",
   "ville":"Rabat","sexe":"homme"},
  {"id":"2","nom":"Safi","prenom":"Amine",
   "ville":"Marrakech","sexe":"homme"}
]

D/ETUDIANT: Etudiant{id=1, nom='Lachgar', prenom='Mohamed',
                     ville='Rabat', sexe='homme'}
D/ETUDIANT: Etudiant{id=2, nom='Safi', prenom='Amine',
                     ville='Marrakech', sexe='homme'}
```

Listing 4.10 – Sortie Logcat après ajout d'un étudiant

Chapitre 5

Challenge — Extension du Projet

5.1 Activité de liste des étudiants

Pour aller plus loin, il est proposé de créer une activité affichant la liste complète des étudiants avec un `RecyclerView`, et permettant la modification et la suppression.

5.1.1 Web Service PHP — deleteEtudiant.php

```php
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    include_once '../service/EtudiantService.php';
    $id = $_POST['id'];
    $es = new EtudiantService();
    $es->deleteById($id);
    header('Content-Type: application/json');
    echo json_encode($es->findAllApi());
}
?>
```

Listing 5.1 – ws/deleteEtudiant.php

5.1.2 Méthode deleteById dans EtudiantService

```php
public function deleteById($id) {
    $stmt = $this->connexion->getConnexion()
                  ->prepare("DELETE FROM Etudiant WHERE id = :id
                    ");
    $stmt->execute([':id' => $id]);
}
```

Listing 5.2 – Ajout dans service/EtudiantService.php

5.1.3 Adapter RecyclerView Android

```java
public class EtudiantAdapter
        extends RecyclerView.Adapter<EtudiantAdapter.ViewHolder
           > {

    private List<Etudiant> etudiants;
    private OnEtudiantActionListener listener;

    public interface OnEtudiantActionListener {
        void onDelete(Etudiant e);
        void onEdit(Etudiant e);
    }

    public EtudiantAdapter(List<Etudiant> etudiants,
                           OnEtudiantActionListener listener) {
        this.etudiants = etudiants;
        this.listener  = listener;
    }

    @Override
    public ViewHolder onCreateViewHolder(ViewGroup parent, int
       viewType) {
        View v = LayoutInflater.from(parent.getContext())
                   .inflate(R.layout.item_etudiant, parent,
                      false);
        return new ViewHolder(v);
    }

    @Override
    public void onBindViewHolder(ViewHolder holder, int
       position) {
        Etudiant e = etudiants.get(position);
        holder.tvNom.setText(e.toString());
        holder.btnDelete.setOnClickListener(v -> {
            new AlertDialog.Builder(v.getContext())
                .setTitle("Confirmation")
                .setMessage("Supprimer cet etudiant ?")
                .setPositiveButton("Oui", (d, w) -> listener.
                   onDelete(e))
                .setNegativeButton("Non", null)
                .show();
        });
    }

    @Override
    public int getItemCount() { return etudiants.size(); }

    static class ViewHolder extends RecyclerView.ViewHolder {
        TextView tvNom;
        Button   btnDelete;
        ViewHolder(View v) {
            super(v);
```

```
            tvNom      = v.findViewById(R.id.tvNom);
            btnDelete = v.findViewById(R.id.btnDelete);
        }
    }
}
```

Listing 5.3 – EtudiantAdapter.java (squelette)

Chapitre 6

Conclusion

Ce TP a permis de mettre en pratique l'intégralité de la chaîne de développement d'une application mobile connectée :

1. **Base de données MySQL** : création d'un schéma relationnel simple et insertion de données initiales.
2. **Web Services PHP** : développement d'une API REST légère en PHP, exploitant PDO pour des requêtes sécurisées et retournant des données JSON.
3. **Application Android** : consommation des Web Services via Volley, désérialisation JSON avec Gson, et affichage des résultats dans Logcat.

Les concepts clés maîtrisés à l'issue de ce TP sont les **requêtes HTTP asynchrones** avec Volley, le **parsing JSON** avec Gson, la **connexion PDO** en PHP, ainsi que la **configuration de sécurité réseau** sur Android.

Pour aller plus loin

Les prochaines étapes naturelles seraient : sécuriser l'API PHP avec une authentification par token JWT, afficher la liste dans un `RecyclerView` sur Android, et implémenter les opérations de mise à jour et de suppression.